SDSC Summer Institute 2026

Performance Tuning

Igor Sfiligoi

Performance tuning

In this session,
we explore options to get more performance out of your tools.

Said another way, we look for ways to either

- Get the same problem solved faster.
- Get more problems solved in the same amount of time.

Why?

Why do you care?

- Getting results faster is of course always welcome
  - But does is make a material difference in real life?

Why do you care?

- Getting results faster is of course always welcome
  - But does is make a material difference in real life?
- For example:
  A tool takes 600ms to complete, and you reduce that to 10ms
  - If it is rarely used more than a few times a day, nobody will care
  - If it is something that is executed every time a process starts,
    it will likely make a noticeable cumulative difference
- Similarly, reducing a function runtime from 600ns to 10ns
  - If it is invoked once per process, it is unlikely anyone will notice
  - If it is invoked billions of times in an inner loop, it will be a game-changer

Why do you care?

- Getting results faster is of course always welcome
  - But does is make a material difference in real life?
- For example:
  A tool takes **1 hour** to complete, and you reduce that to **1 minute**
  - Even if rarely used, it will likely make a big difference

- Similarly, reducing a function runtime from **1 minute** to **1 second**
  - Even if it is invoked once per process, it will likely make a difference

Cost/benefit analysis

Getting results faster rarely comes for free

- Understand the cost
  - Are you paying it once?
  - Or will you have to pay many times to maintain the speedup?
- Are you able to pay the cost?
  - It may be expressed in either money or needed effort (or both)
- Are you willing to pay the cost?
  - Is it a nice to have?
  - Or is it transformational?
    - Without the speedup, it may take years to get the result!
    - One hour per iteration is doable. But I get distracted. A 1s reply allows for exploration.

Can I just use different HW?

HW vs SW

- Making software faster is obviously ideal
  - If done right, it is a **one time cost**
  - But **human time is expensive**
    (Modern AI tools can help, but they are not a silver bullet)
  - And may not always be possible
- Using different hardware to get results faster usually easier
  - Could even be cheaper
    - Especially if it is a one-off endeavor
  - But it is **a recurring cost**
    (i.e. it will add up over long periods of time)
  - And may still require some software changes

Types of hardware resources

- Datacenter vs consumer hardware
  - A high-end node in a datacenter is likely 10x to 100x faster than a laptop
  - You may be able to use identical software
- Clusters vs single nodes
  - Clusters can scale to 1000x of nodes
  - Pleasantly parallel problems may get 1000x faster (without any changes)
  - But most other problems will not scale linearly,
    and will likely hit a wall at one point.
    Plus, some software changes may be needed.
- Exotic hardware may provide even bigger speedups
  - E.g. GPUs, FPGAs, dedicated accelerators
  - Likely require major changes to the software

Can my SW be made faster?

Can SW be made faster?

- As we said, improving SW speed ideal
  - But not all SW can be made faster (on identical HW)

- First, you must **understand where your SW spends its time**
  - Both logical analysis and actual measurement recommended
  - A measurement-based approach is usually called **profiling**
- If 90% of the time is spent in a very small portion of the code, you better speed up that part!
  - This would be called **a hotspot**
- If compute is uniformly spread, it is very hard to get any improvement

Measure the right thing

- Use a realistic use case
  - Toy problems may not be representative
- Use the fully optimized executable
  - Compilers only do the right thing there (performance-wise)
  - Never benchmark a debug build
- Use realistic hardware setup
  - Using a single CPU core not the same as using the whole CPU (all cores)
  - A laptop CPU may behave differently than a datacenter one

Profiling modes

- Four major approaches
  - Timers in user code
  - Counting profilers
  - Sampling profilers
  - Hardware counters

Profiling modes

- Four major approaches
  - **Timers in user code**
  - Counting profilers
  - Sampling profilers
  - Hardware counters

The code owner is in the best position to understand the logical phases of an application.

Having a **coarse-grained idea of where** the applications spends its time is a great start!
And easy to measure how it changes with time and problem, as it is always on.

But usually needs to be paired with other methods.

Profiling modes

- Four major approaches
  - Timers in user code
  - **Counting profilers**
  - Sampling profilers
  - Hardware counters

Compilers can instrument the application, and **automatically insert** invocation counters and timing counters into your executable.

Gives you the **complete description** of the application's internal workflow, and how execution time is spent.

Unfortunately, this tends to add a non-negligible overhead, both slowing down the execution and **biasing the results**.

Profiling modes

- Four m
  - Time
  - **Cou**
  - Sam
  - Hard

```
   Incl | Incl Time |      Loop Exec |    Loop | Calltree=/[.]LOOP[.]
  Time% |           |                |   Trips |
        |           |                |     Avg |

 100.0% | 22,513.44 |             -- |      -- | Total
|-----------------------------------------------------------------------
1  99.9% | 22,481.98 |              1 | 7,001.0 | smain_.LOOP.8.li.335
||----------------------------------------------------------------------
2|  96.9% | 21,809.35 |          7,000 | 7,855.0 | stiff_ulgr_.LOOP.59.li.1905
|||---------------------------------------------------------------------
3||  89.9% | 20,231.88 |     54,985,000 |   134.3 | ulagra_.LOOP.1.li.81
||||--------------------------------------------------------------------
4|||   4.4% |    981.01 | 14,770,952,000 |     3.0 | mulqab_.LOOP.1.li.140
|||||-------------------------------------------------------------------
5||||   1.9% |    420.54 | 44,312,856,000 |     3.0 | mulqab_.LOOP.2.li.142
||||====================================================================
3||   1.0% |    214.79 |    423,696,000 |     3.0 | mulpru_.LOOP.1.li.85
||||--------------------------------------------------------------------
4|||   0.4% |     84.43 |  1,271,088,000 |     3.0 | mulpru_.LOOP.2.li.86
|||||-------------------------------------------------------------------
5||||   0.2% |     36.16 |  3,813,264,000 |     3.0 | mulpru_.LOOP.3.li.88
||||====================================================================
3||   0.7% |    155.40 |     54,985,000 |   134.3 | stiff_ulgr_.LOOP.60.li.1943
3||   0.6% |    142.89 |     54,985,000 |   134.3 | stiff_ulgr_.LOOP.83.li.2575
```

cation, and
unters and
e.

n of the

ng down the

Profiling modes

- Four major approaches
  - Timers in user code
  - Counting profilers
  - **Sampling profilers**
  - Hardware counters

You basically take a snapshot of where the code is many times during the execution of an application.

Taking a snapshot is extremely cheap, so it does not affect the performance of your application, resulting in an **unbiased measurement**.

It is a **great way to find hotspots**.
But does not help you identify the workflow that led there.

Profiling modes

- Four major ap
  - Timers in us
  - Counting pr
  - **Sampling p**
  - Hardware c

```
  Samp% |      Samp | Group
        |           |  Function

 100.0% | 263,922.0 | Total
|-------------------------------------
|  76.8% | 202,753.0 | USER
||------------------------------------
||  48.6% | 128,384.0 | mulqab_
||  14.6% |  38,557.0 | stiff_ulgr_
||   4.4% |  11,500.0 | mulpru_
||   3.3% |   8,671.0 | ulagra_
||   1.2% |   3,212.0 | msurfpair_
||   1.2% |   3,046.0 | interpolate_all_
||====================================
|  11.1% |  29,199.0 | HEAP
||------------------------------------
||  10.7% |  28,357.0 | calloc
||====================================
|   5.2% |  13,839.0 | BLAS
||------------------------------------
||   4.4% |  11,722.0 | _dgemm_
||====================================
|   2.7% |   7,220.0 | ETC: libf.so.2.0
||------------------------------------
```

ke a snapshot of where the code is
ring the execution of an application.

not is extremely cheap, so it
the performance of your application,
unbiased measurement.

y to find hotspots.
lp you identify the workflow

Profiling modes

- Four major approaches
  - Timers in user code
  - Counting profilers
  - Sampling profilers
  - **Hardware counters**

Real hardware is composed of many different parts. It is often necessary to understand which part is your application stressing the most.

Most systems provide access to hardware-maintained counters. They may not be specific to your application, but are still representative if you are the only one stressing the system.

Exercise time

- Log into Expanse
- Checkout the git repository, if you did not do it already
- Go into 5.2_performance_tuning/exercises
- Do the exercise:
  - 1_profiling

Is there a better way to solve it?

You found a hotspot, now what?

- The best performance improvements usually come from better algorithms
  - I.e., Can you find a better way to solve the same problem?
- No generic advice I can give you there!
  - Unless the original implementation was a Proof of Concept only, it may need **serious research effort**!
  - But do **check if anyone else is doing the same thing**; they may have a better implementation than you do
  - Standard (or vendor-provided) libraries are definitely worth exploring

You found a hotspot, now what?

- The best performance improvements usually come from better algorithms
  - I.e., Can you find a better way to solve the same problem?
- No generic advice I can give you there!
  - Unless the original implementation was a Proof of Concept only, it may need serious research effort!
  - But do check if anyone else is doing the same thing; they may have a better implementation than you do
  - Standard (or vendor-provided) libraries are definitely worth exploring
- We will assume you are stuck with the existing conceptual algorithm in the rest of these slides

Understand your HW

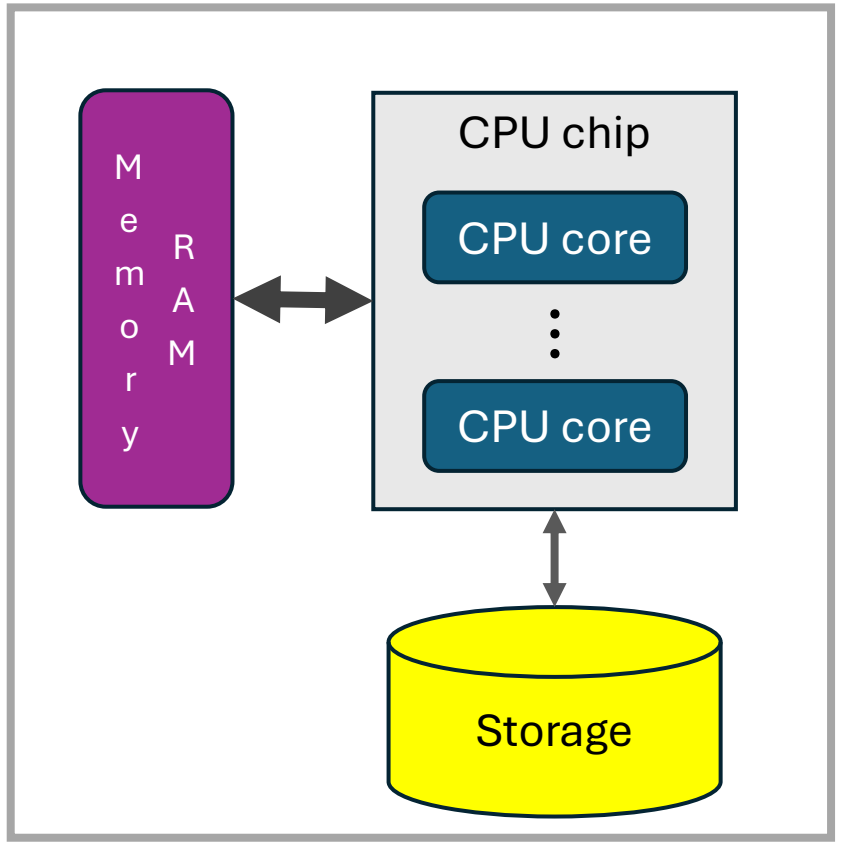
Software just drives the hardware

- The only way to make software faster (without changing the algorithm) is to **use the hardware better**
- But first you must understand
  - What hardware are you using?
  - How that hardware works?
- Spoiler:
  It is not a single, simple item

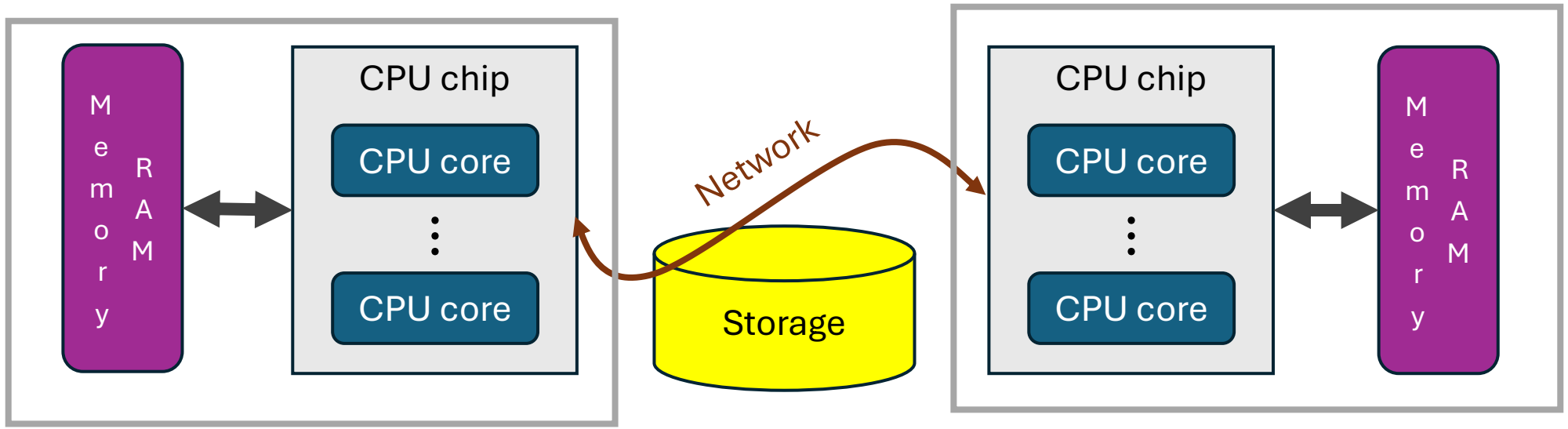
We will focus on CPU-only systems

- Luckily, virtually all CPUs are the same
  - At least conceptually
  - We will ignore some “weirdnesses” inherent to mobile systems
- GPU systems require a slightly different mindset
  - More on GPUs in another session
- FPGAs and dedicated accelerators are a completely different world
  - Many CPU concepts don’t apply there at all!

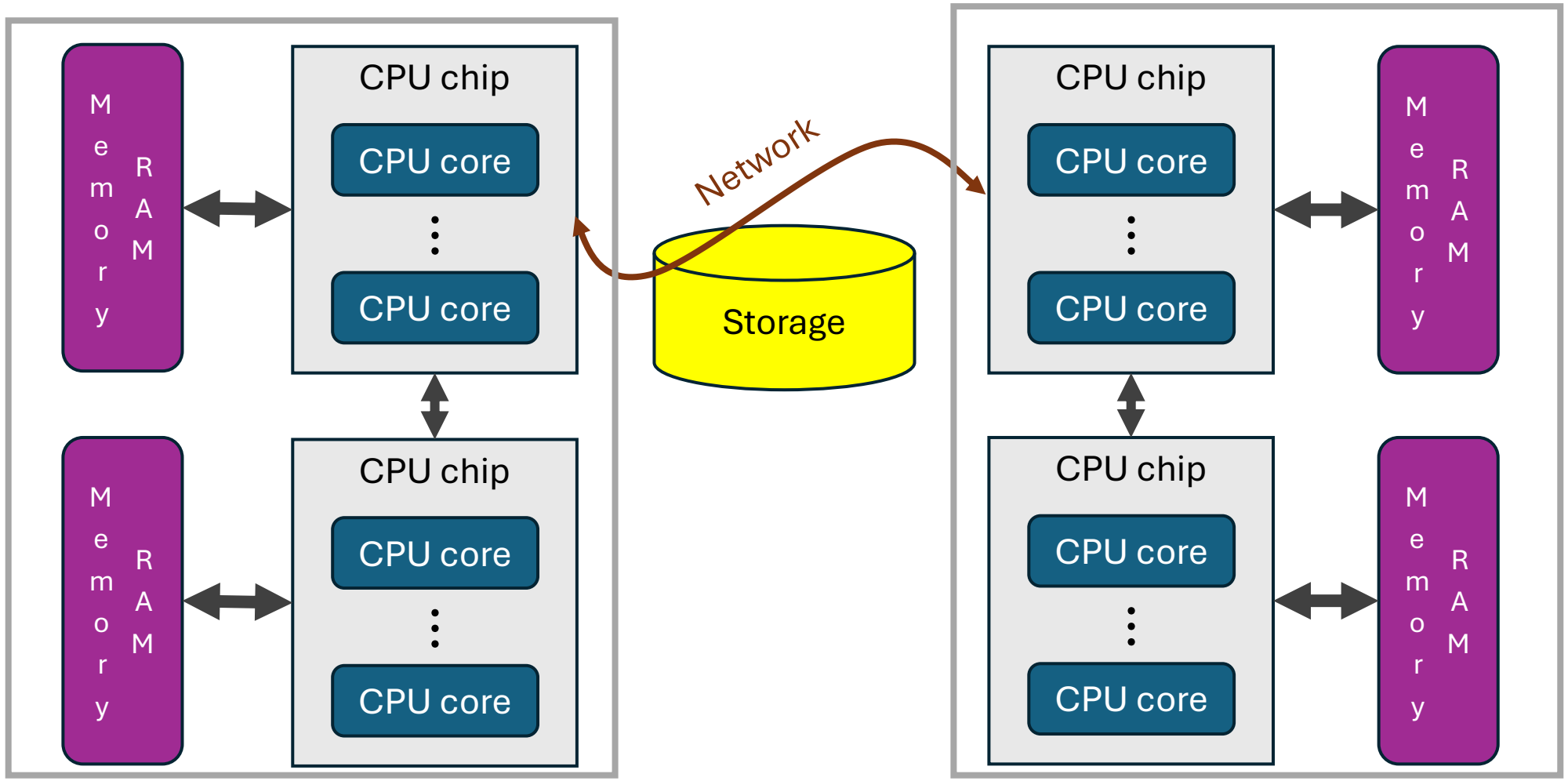
Very high-level view



Very high-level view - HPC



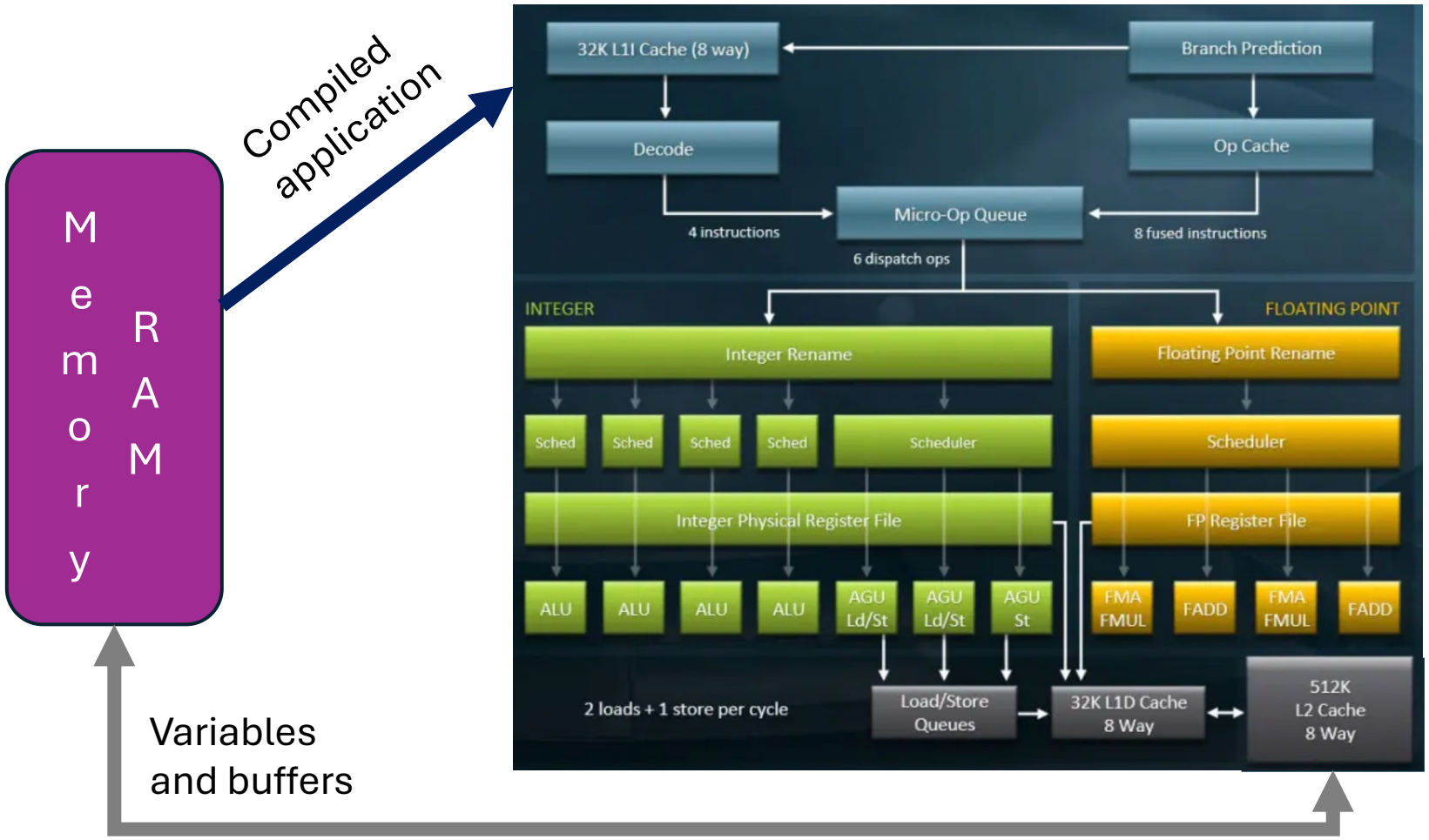
Very high-level view - HPC



Initial insight

- You do not use all the compute capability
  if you do not use all the CPU cores (e.g. OpenMP)
- When using multiple CPU cores,
  those cores compete for memory access
- Not all memory locations are the same
  NUMA = Non-Uniform Memory Access
  - Physically attached is faster
  - Accessing a memory location on another CPU much slower
  - Accessing memory on another node may need
    explicit network network communication (MPI)

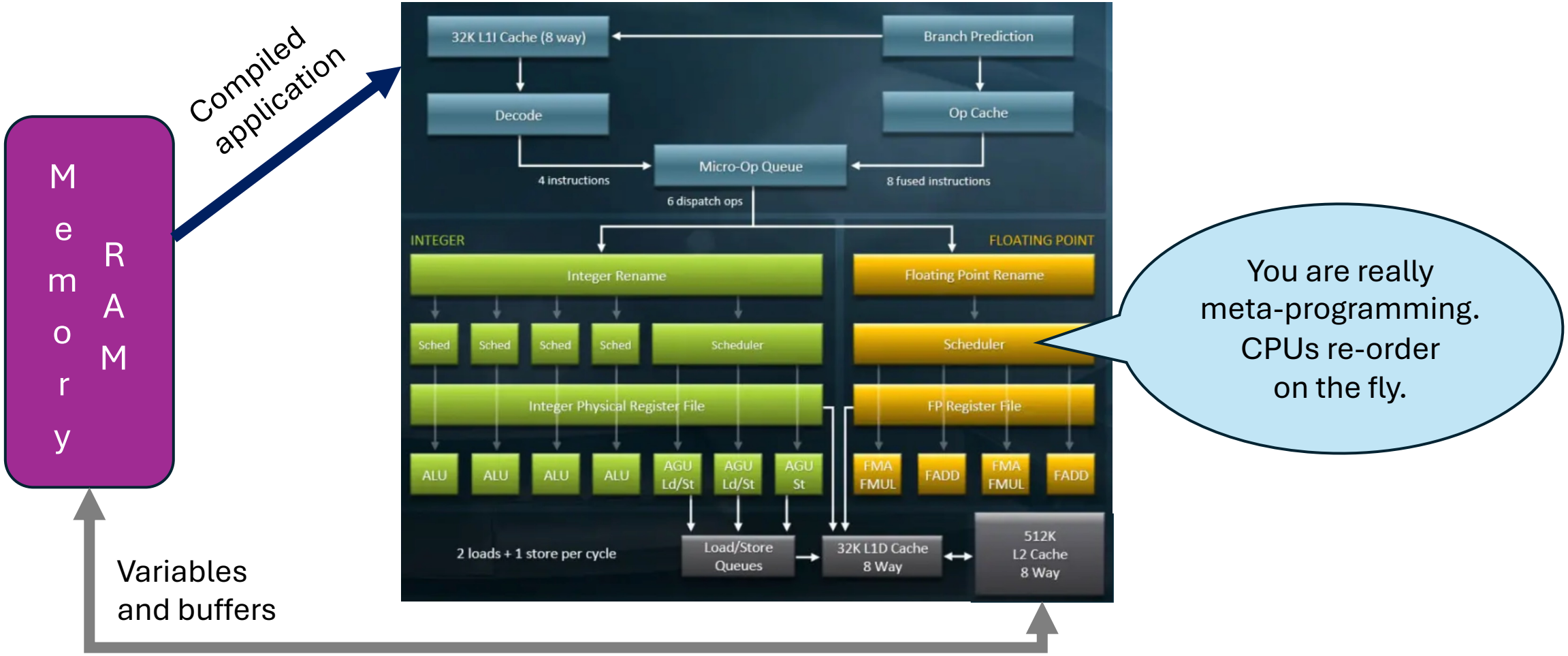
In deep detail - Anatomy of a CPU core



The AMD “Rome” example

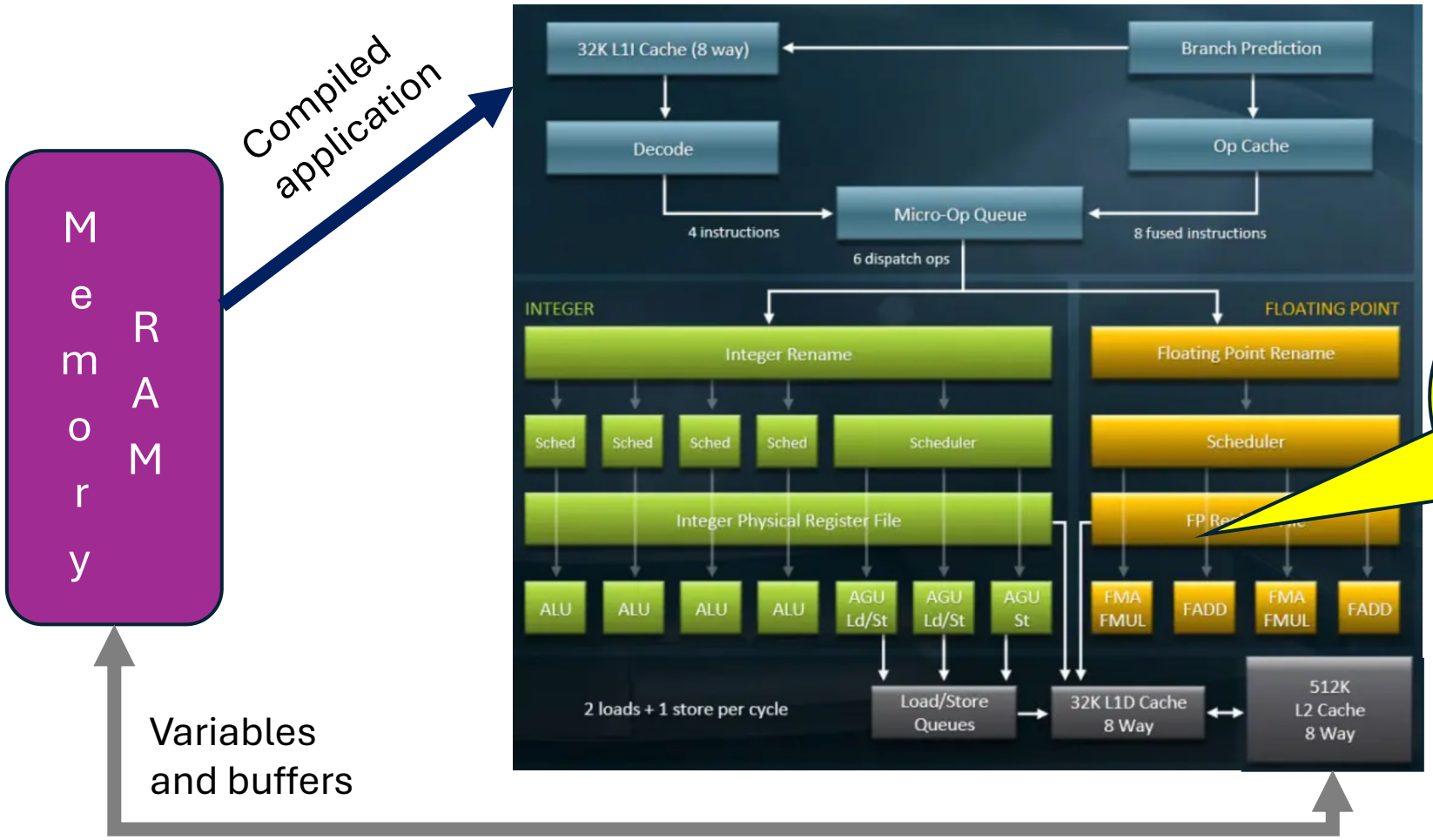
https://www.nextplatform.com/compute/2019/08/15/a-deep-dive-into-amds-rome-epyc-architecture/1659190

In deep detail - Anatomy of a CPU core



https://www.nextplatform.com/compute/2019/08/15/a-deep-dive-into-amds-rome-epyc-architecture/1659190

In deep detail - Anatomy of a CPU core



Need many independent compute steps to use all the compute units

Great:
a=sqrt(x);
b+=y*z;

Bottleneck:
a=sqrt(x);
b+=y*a;

https://www.nextplatform.com/compute/2019/08/15/a-deep-dive-into-amds-rome-epyc-architecture/1659190

In deep detail – Anatomy of a CPU core

Instructions are pipelined;
For most instructions, there is a big difference between how soon the result will be available (Latency) vs how often a new instruction can be scheduled (1/CPI):

Sqrt

Latency and Throughput

Architecture	Latency	Throughput (CPI)
Alderlake	12	6

FMA

Latency and Throughput

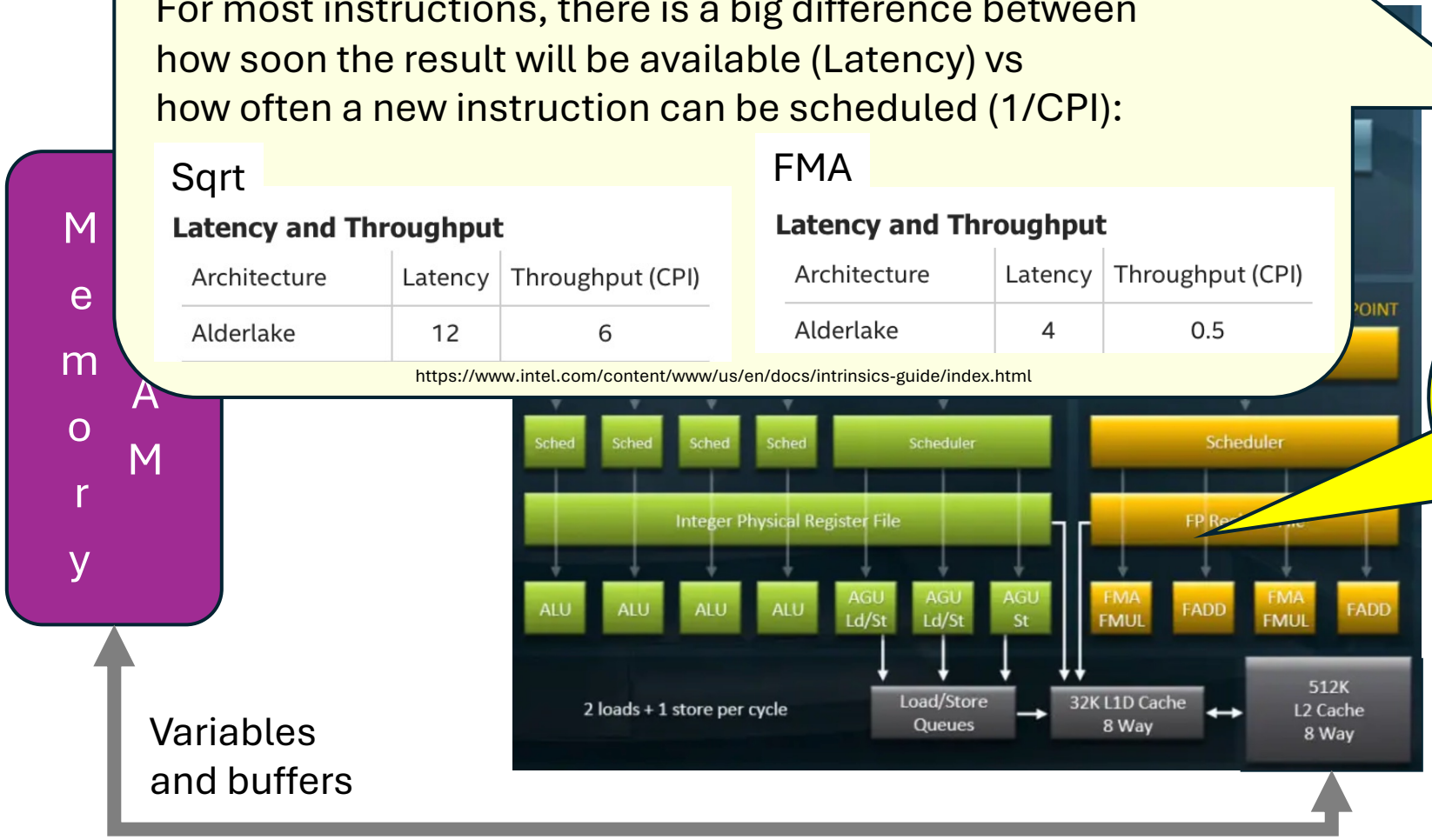
Architecture	Latency	Throughput (CPI)
Alderlake	4	0.5

https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html

Great:
a=sqrt(x);
b+=y*z;

Need many independent compute steps to use all the compute units

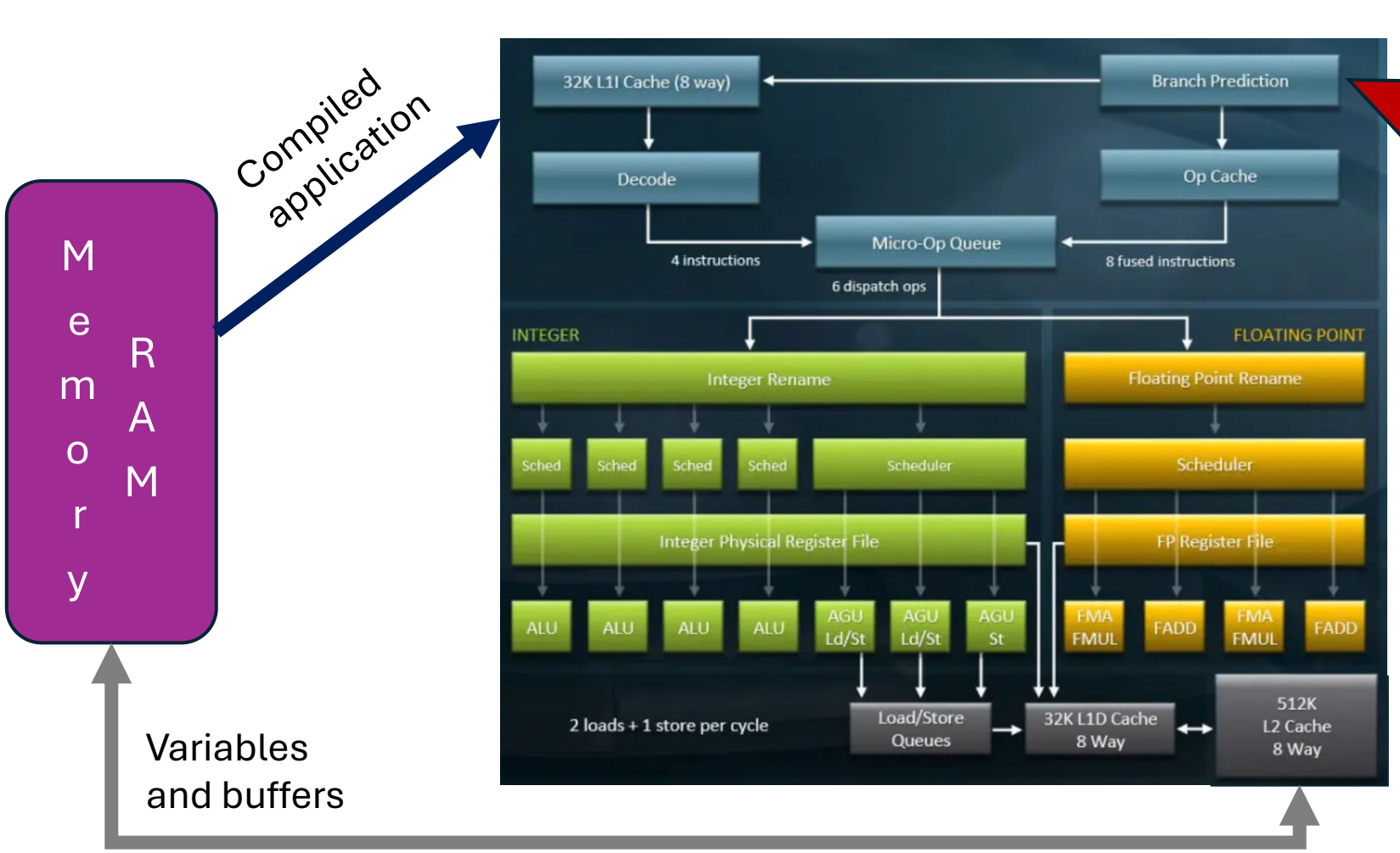
Bottleneck:
a=sqrt(x);
b+=y*a;



https://www.nextplatform.com/compute/2019/08/15/a-deep-dive-into-amds-rome-epyc-architecture/1659190

In deep detail - Anatomy of a CPU core

Good:
a=sqrt(x);
b+=1/sqrt(y);



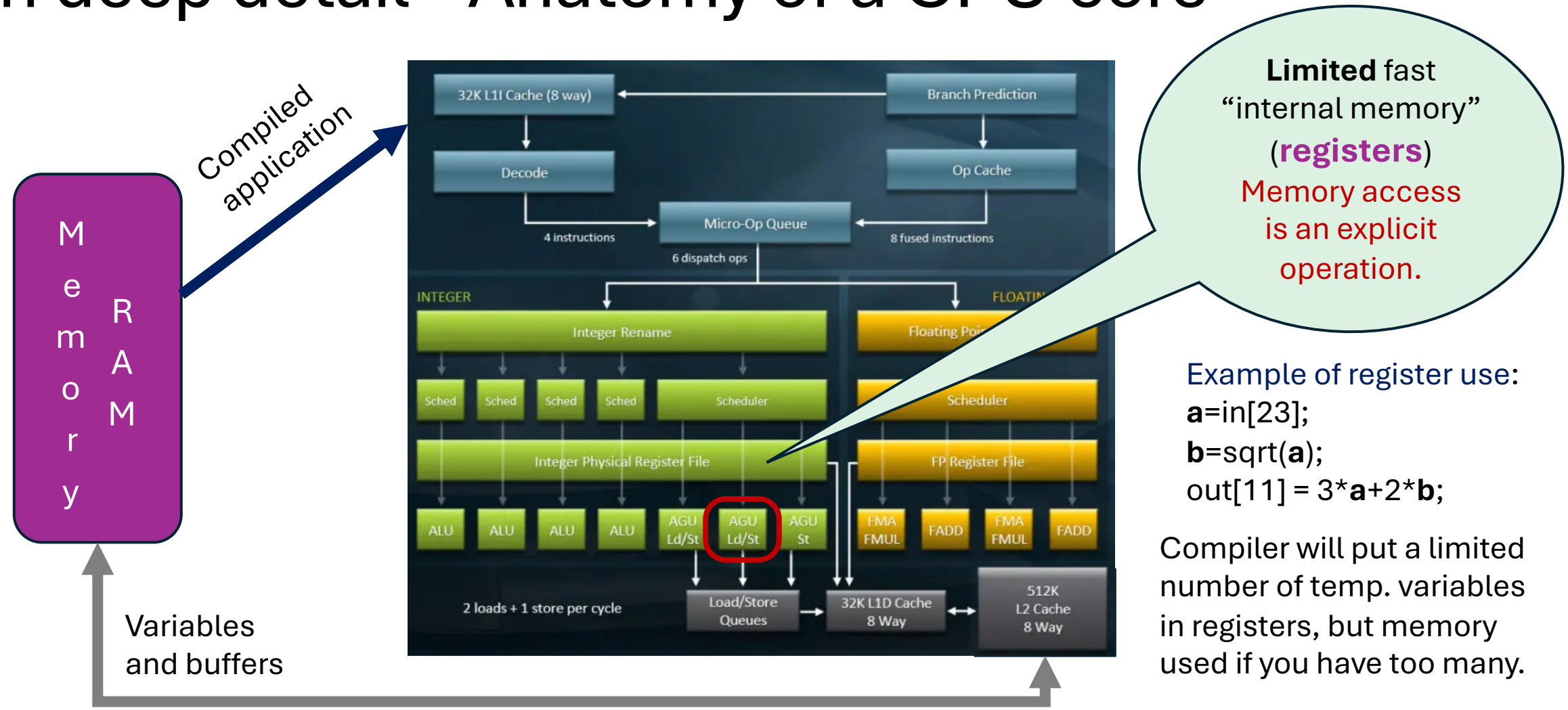
Branching (**if** statements) breaks internal parallelism. **CPU will try to guess.**

Likely OK:
a=sqrt(x);
If (flag1) b+=1/sqrt(y);
else b+=sqrt(z);

Quite bad:
a=1/sqrt(x);
If (a<1) b+=1/sqrt(y);
else b+=sqrt(z);

https://www.nextplatform.com/compute/2019/08/15/a-deep-dive-into-amds-rome-epyc-architecture/1659190

In deep detail - Anatomy of a CPU core



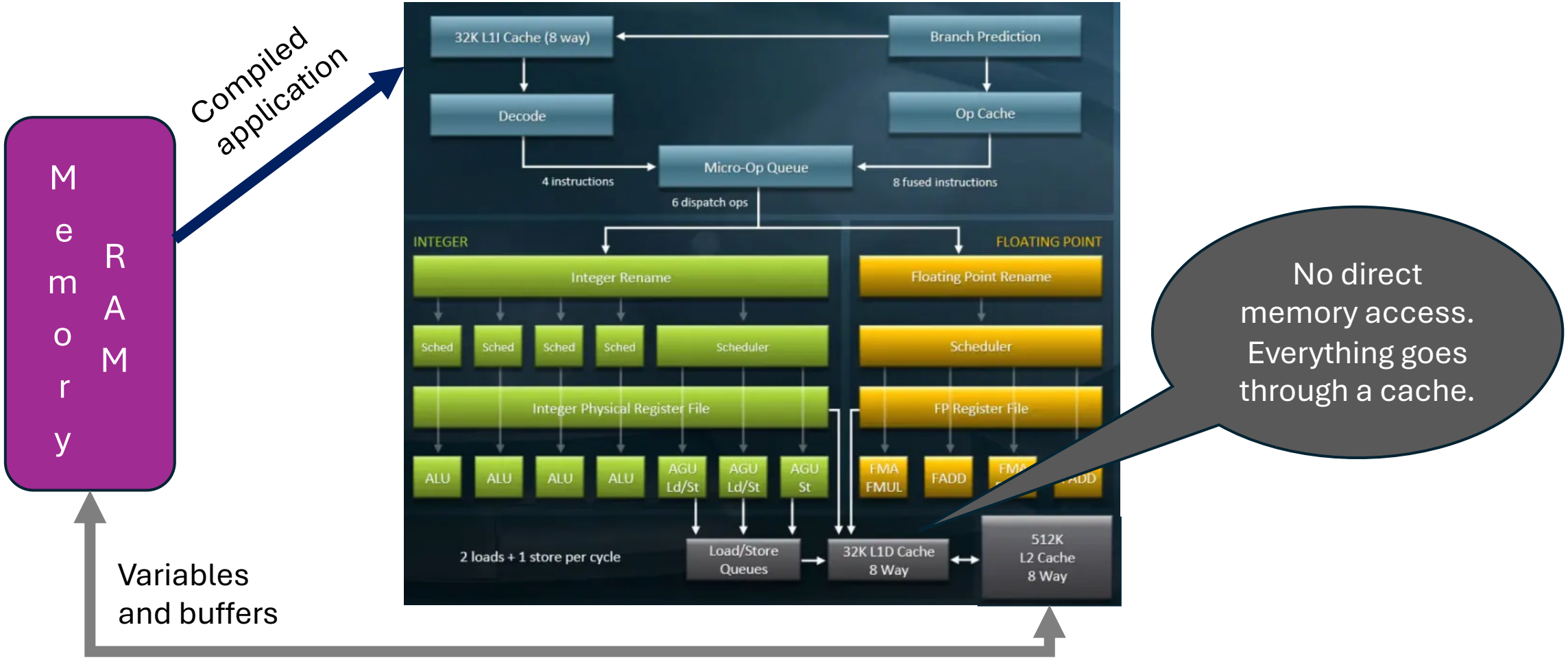
Example of register use:

a=in[23];
b=sqrt(**a**);
out[11] = 3***a**+2***b**;

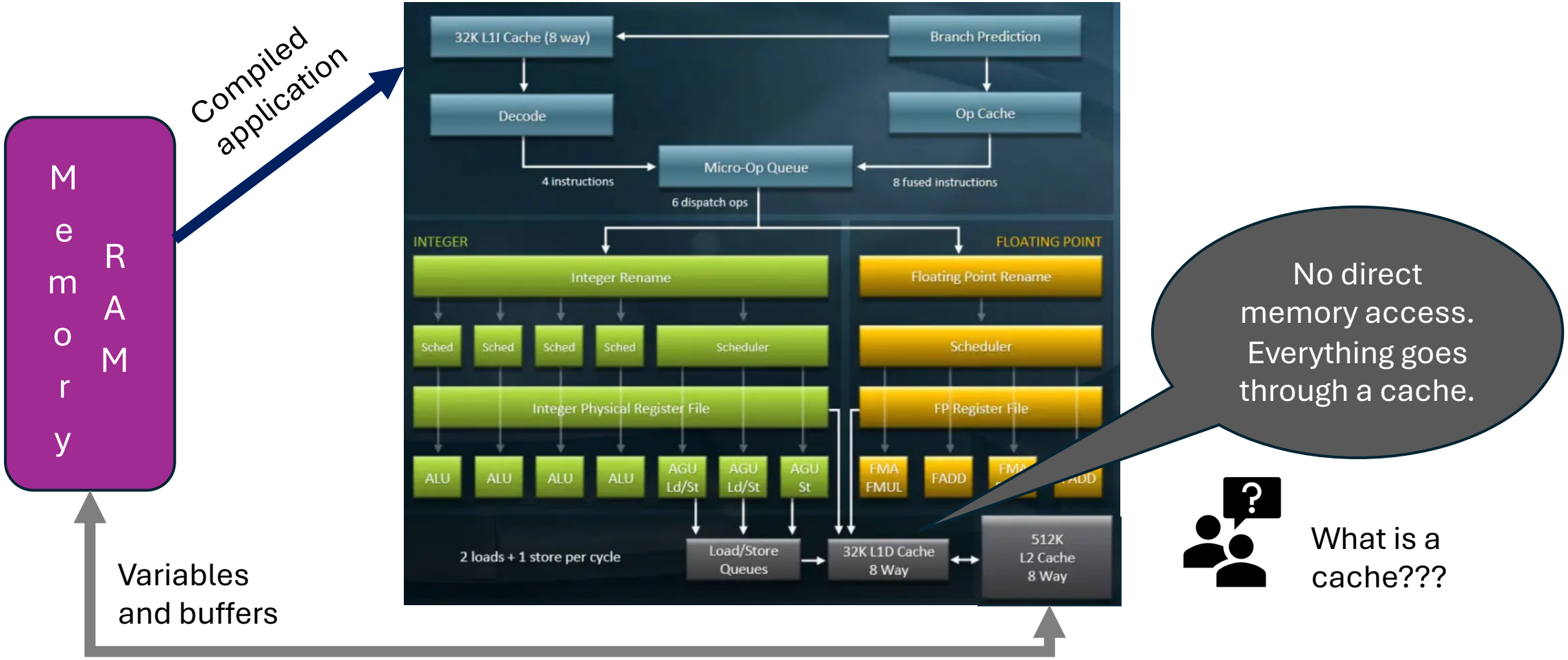
Compiler will put a limited number of temp. variables in registers, but memory used if you have too many.

https://www.nextplatform.com/compute/2019/08/15/a-deep-dive-into-amds-rome-epyc-architecture/1659190

In deep detail - Anatomy of a CPU core



In deep detail - Anatomy of a CPU core



The cost of direct memory access

- Accessing information straight from memory is very expensive
  - You can do about **300 compute operations** in the time it takes to fetch a byte from main memory (RAM)!
- Classic examples, like AXPY, can only go as fast as memory can deliver
  - Compute is basically free there

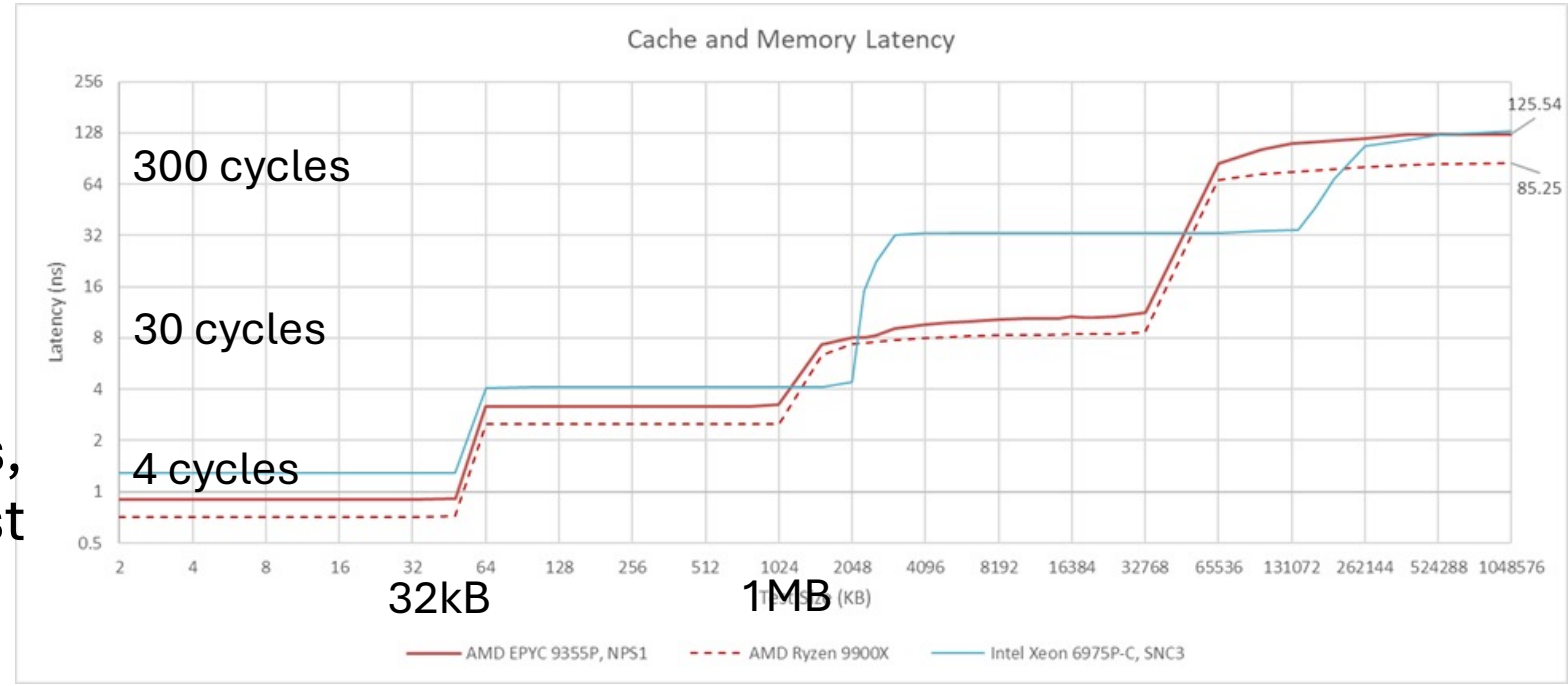
AXPY:
x[i] += a*Y[i]

Caches to the rescue

- Accessing information straight from memory is very expensive
  - You can do about **300 compute operations** in the time it takes to fetch a byte from main memory (RAM)!
- All modern CPUs thus contain some kind of **memory cache**
  - Access to data in the cache much faster
  - You typically waste **only 4-40 compute operations** (still not free, though)
  - Contains **most recently used data**
    No need to explicitly populate it,
    and data automatically purged to make space

Caches to the rescue

- Accessing information straight from memory is very expensive
- All modern CPUs thus contain some kind of **memory cache**
  - Access to data in the cache much faster
- Cache sizes are small (measured in **kBytes**)
  - Typically several levels, fastest level is smallest



The importance of memory reuse

- The only way to fully utilize the compute capabilities of modern CPU cores, is to **reuse the same data over and over again**
  - Ideally from registers
  - But L1 cache can be good enough if you can use internal parallelism
  - Everything else is memory-bound!
- Basic advice:
  - **Avoid** accessing the same buffer multiple times!
  - Try to extract all the information in one pass

Bad:

```
for i: x[i] = y[i]+z[i]
for i: x[i] += sin(y[i])
for i: x[i] = min(x[i],sqrt(z[i])
```

Great:

```
for i:
  a=y[i]; b=z[i]
  x[i]=min(a+b+sin(a),sqrt(b))
```

The importance of memory reuse

- The only way to fully utilize the compute capabilities of modern CPU cores, is to **reuse the same data over and over again**
  - Ideally from registers
  - But L1 cache can be good enough if you can use internal parallelism
  - Everything else is memory-bound!
- Basic advice:
  - Avoid accessing the same buffer multiple times!
  - If you need a **work buffer, keep it small**.
    - So it fits in L1 cache (**< 24kB** on most systems)
    - May not always be possible, but think hard if there is a way (Can you use a smaller el. type? fp32 vs fp64? bool vs int?)

Good:

```
for (s,e) in split_range(N,1k):
  x[s:e] = y[s:e]+z[s:e]
  x[s:e] += sin(y[s:e])
  x[s:e] = min(x[s:e],sqrt(z[s:e])
```

Understand the cache line

- Cache granularity is **not** at the byte level
  - A **cache line** is typically 64 bytes
  - The whole cache line is present in a cache, or none of its bytes are!
- **Strive for contiguous memory access**
  - Else, your effective work area explodes!

Example where “small index loop” does not fit in L1 cache.

```
struct MyEl {
  double d[7]; // 8*7 bytes
  bool f[7]; // 7 bytes
}; // 64 bytes total (1 wasted)

double sum(int i, int j, MyEl  b[1024]) {
  for el in b: el.d[0] = 0;
  for el in b: if (el.f[i]) el.d[0] += el.d[i];
  for el in b: if (el.f[j]) el.d[0] += el.d[j];
  double sum = 0;
  for el in b: sum += el.d[0];
  return sum;
};
```

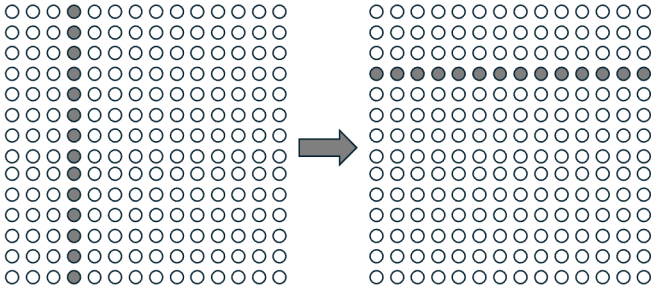
Understand the cache line

- Cache granularity is **not** at the byte level
  - A **cache line** is typically 64 bytes
  - The whole cache line is present in a cache, or none of its bytes are!
- Strive for contiguous memory access
- When writing, be careful of **false sharing**
  - If two cores access the **same** cache line, only one can own it
  - A write will always invalidate cache lines in other CPU cores (ping-pong effect)

Example of false sharing:

```
int cnt[...];
#pragma omp parallel
for (int i=0; i<N; i++) {
  if (sqrt(x[i]<1) cnt[omp_get_thread_num()]++;
}
```

Useful technique – 2D tiling



- Classic transpose example

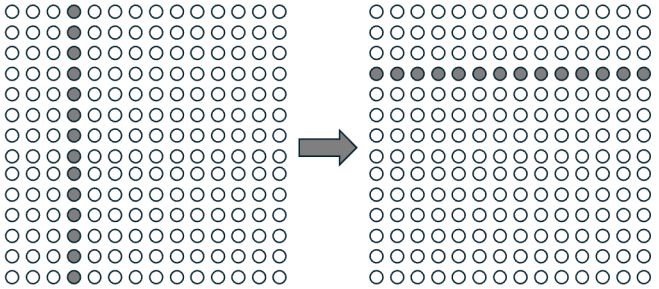
Naïve:

```
for (int i = 0; i < N; i++)
 for (int j = 0; j < M; j++) {
   B[i * M + j] = A[j * N + i];
 }
```

Reads a whole column.

We populate 64*M bytes in L1 cache.
If M is small, the next 7 iterations
will reuse the data from cache.
(Assuming double elements, i.e. 8 bytes)

Useful technique – 2D tiling



- Classic transpose example

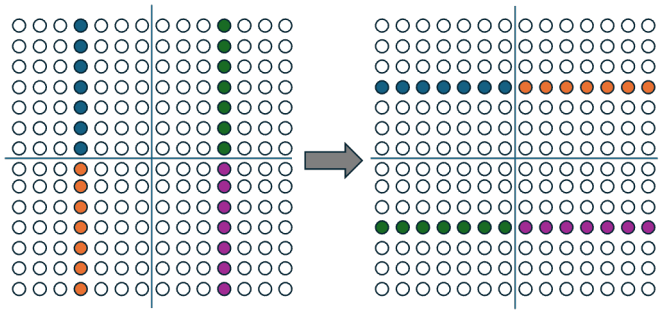
Naïve:

```
for (int i = 0; i < N; i++)
 for (int j = 0; j < M; j++) {
   B[i * M + j] = A[j * N + i];
 }
```

Reads a whole column.

We populate 64*M bytes in L1 cache.
If M is large (>500), we are reading those same 64*M bytes from slower memory again!

Useful technique – 2D tiling



- Classic transpose example

Transpose one small sub-matrix at a time.

Guaranteed to read from the same cache line.

Reads only T elements.

Naïve:

```
for (int i = 0; i < N; i++)
 for (int j = 0; j < M; j++) {
  B[i * M + j] = A[j * N + i];
 }
```

Tiled: (T decided at compile time)

```
for (int sj = 0; sj < N; sj += T)
 for (int sj = 0; sj < M; sj += T) {
  for (int i = si; i < si + T; i++)
   for (int j = sj; j < sj + T; j++) {
    B[i * M + j] = A[j * N + i];
   }
 }
```

(Assuming N and M divisible by T)

We access 64*T bytes in the first pass. By **picking** a small T, the next 7 passes will already have the data in L1 cache.
(Assuming double elements, i.e. 8 bytes)

Useful technique – 2D tiling

- Classic transpose example

Naïve:

```
for (int i = 0; i < N; i++)
 for (int j = 0; j < M; j++) {
  B[i * M + j] = A[j * N + i];
 }
```

Tiled:

```
for (int sj = 0; sj < N; sj += T)
 for (int sj = 0; sj < M; sj += T) {
   for (int i = si; i < si + T; i++)
     for (int j = sj; j < sj + T; j++) {
        B[i * M + j] = A[j * N + i];
     }
 }
```

Using OpenMP:

```
#pragma omp tile sizes(T,T)
for (int i = 0; i < N; i++)
 for (int j = 0; j < M; j++) {
  B[i * M + j] = A[j * N + i];
 }
```

Same semantics.
Tiling happens under the hood.
(Requires OpenMP 5.1+ compiler)

The first access problem

- Caches are great for speeding up (small) temp buffers
  - I.e., Reading back recently written data
- But most data not already in cache
  - Caches can still help!
  - Most CPUs use **prefetching**
    I.e. the CPU will guess what the next needed cache line will be and fetches it into Cache before you ask (in background)
  - If compute/memory ratio large, may make memory access free
- Prefetching works great for sequential access
  - E.g. AXPY-like

The first access problem

- Caches are great for speeding up (small) temp buffers
  - I.e., Reading back recently written data
- But most data not already in cache
  - Caches can still help!
  - Most CPUs use **prefetching**
- Prefetching works great for sequential access
- Does not work for pseudo-random access
  - For example, tree traversal
  - You can help by using explicit prefetch commands
    In gcc, it is called **__builtin_prefetch**

Sharing data between CPU cores

- Most modern CPU share a L3 cache among several cores
  - Much slower than L1 cache, but still significantly faster than main RAM
- When using parallel programming (e.g. OpenMP)
  - If your **read-only work area** is too big for L2, try at least to **fit it in L3 Cache**
  - Size of L3 cache often comparable to sum of L2 cache sizes, but you do not need to partition it
- Note: Not helpful for memory you write to

Modern AMD Epyc CPU

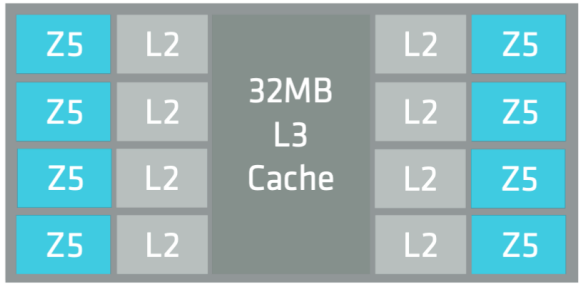


Figure 2: A 'Zen 5' CCD with 8 cores

https://docs.amd.com/v/u/en-US/5th-gen-amd-epyc-processor-architecture-white-paper

All modern CPUs do vector math

- Not emphasized in the previous diagrams is the fact that **all compute units are vectorized**!
- **Computing a 256-bit vector takes as much time as computing an 8-bit scalar**
  - Scalars are really just partially filled vectors, from a HW point of view
- Only way to make full use of CPU compute capabilities is write vector code

Scalar int32 mul		Vector int32x4 mul	
imul		**vpmuldq**	
Latency	CPI	Latency	CPI
3	0.33	3	0.5

On Zen 5, https://uops.info/table.html

How do we write vector code?

- For simple loops, the compiler takes care of everything
  - E.g. AXPY
- For more complex loop, compiler can still do the right thing, IF:
  - You **access consecutive memory locations**
  - Have no complex logic (if statements)

Likely vectorized:

```
for i:
  a=y[i]; b=z[i]
  x[i]=min(a+b+sin(a),sqrt(b))
```

Probably not vectorized:

```
for i:
  a=y[i*3]; b=z[i%7]
  if (w[i%25]>sin(b)) a+=b;
  x[i*3]=min(a+b+sin(a),sqrt(b))
```

https://gcc.gnu.org/onlinedocs/gcc-8.3.0/gcc/Vector-Extensions.html

How do we write vector code?

- For simple loops, the compiler takes care of everything
  - E.g. AXPY
- For more complex loop, compiler can still do the right thing, IF:
  - You **access consecutive memory locations**
  - Have no complex logic (if statements)
- For really complex logic, you may need to use explicit vector types
  - Not part of most language specs
  - But most modern compilers have native support for it

GCC example:

```
typedef int v4si __attribute__ ((vector_size (16)));

v4si a, b, c;

c = a + b;
```

https://gcc.gnu.org/onlinedocs/gcc-8.3.0/gcc/Vector-Extensions.html

How do I know if the code was vectorized?

- Most compilers have compile options to tell you if a loop was vectorized
  - Use **-fopt-info-vec** with gcc

```
$ gcc -O3 -fopt-info-vec -o axpy_o3 axpy.c -lm
axpy.c:19:5: optimized: loop vectorized using 16 byte vectors
axpy.c:93:5: optimized: loop vectorized using 16 byte vectors
axpy.c:19:5: optimized: loop vectorized using 16 byte vectors

$ gcc -O3 -fopt-info-vec -march=native -o axpy_o3 axpy.c -lm
axpy.c:19:5: optimized: loop vectorized using 32 byte vectors
axpy.c:93:5: optimized: loop vectorized using 32 byte vectors
axpy.c:19:5: optimized: loop vectorized using 32 byte vectors
```

- Too verbose to have it always on
  - But great for when developing now code or trying to optimize existing code
  - Worth checking across compilers and versions, too

How do I know if the code was vectorized?

- Most compilers have compile options to tell you if a loop was vectorized
  - Use **-fopt-info-vec** with gcc
- You can also look at the generated assembly
  - Use **-g -fverbose-asm -save-temps** with gcc
  - Look for **.s** files
  - Usually lots of helper code, too, so may not be trivial to find what you look for

```
$ gcc -O3 -g -fverbose-asm -save-temps -o axpy axpy.c –lm
$ more axpy.s
…
# axpy.c:20:       Z[i] = alpha * X[i] + Y[i];
     vmovups (%rdx,%rax), %ymm1
     vfmadd213ps    (%rcx,%rax), %ymm2, %ymm1
     vmovups %ymm1, (%rsi,%rax)
# axpy.c:19:    for
     addq    $32, %rax
     cmpq    %r8, %rax
     jne    .L5
```

Function calls and vectorization

- Complex/expensive functions should be vectorized on their own
  - Using any of the methods mentioned before
- But **cross-function vectorization may be more effective**
  - **Especially for simple functions**
    (but not only)

```
double rsqrtsum(double a, double b, double m) {
  return 1/sqrt(a)+m*1/sqrt(b);
}
...
for i:
  Z[i] = rsqrtsum(A[i], B[i], C[i]);
```

Function calls and vectorization

- Complex/expensive functions should be vectorized on their own
  - Using any of the methods mentioned before
- But **cross-function vectorization may be more effective**
  - **Especially for simple functions** (but not only)
- Compilers can only vectorize if they see the function content
  - A problem with separate compilation
  - Use **header-only inline** in C++, when possible

```
inline double rsqrtsum(double a, double b, double m) {
  return 1/sqrt(a)+m*1/sqrt(b);
}
...
for i:
  Z[i] = rsqrtsum(A[i], B[i], C[i]);
```

Function calls and vectorization

- Complex/expensive functions can be vectorized on their own
  - Using any of the methods mentioned before
- But **cross-function vectorization may be more effective**
  - **Especially for simple functions** (but not only)
- Compilers can only vectorize if they see the function content
- You can also use explicit vectorization at the interface

```
typedef double v4d __attribute__ ((vector_size (32)));
v4d rsqrtsum(v4d &a, v4d &b, v4d &m) {
  return 1/sqrt(a)+m*1/sqrt(b);
}
...
for i:
  Z[i] = rsqrtsum(A[i], B[i], C[i]);
```

Use all the CPU cores

- No single-core CPUs anymore
  - You must find a way to parallelize your problem
- You can do it in three orthogonal ways
  - Split in several independent subproblems (with minimal glue before and after)
    - You had an introduction to HTC a few days ago
  - Partition the problem in overlapping sub-problems
    - We discussed how to use MPI to deal with synchronization
  - Distribute the (sub-)problem over multiple cores
    - But remember NUMA and race conditions
    - We went over OpenMP a few days ago
- A combination of the 3 will give you the best results

Use all the CPU cores

- No single-core CPUs ap
  - You must
- You can
  - Spli
  - I
    -
  - D
    -
    - We wer
- A combination of the 3 wit e best results

But remember:
Main memory (and L3 cache) are shared
between CPU cores,
so the more CPU cores you use,
the slower access to memory will be.
(per core)

and after)

Exercise time

- Log into Expanse
- Checkout the git repository, if you did not do it already
- Go into 5.2_performance_tuning/exercises
- Do the exercises:
  - 2_memory
  - 3_vectors
  - 4_functions
- You can also try to optimize code from other sessions

In summary

- Make sure you understand where the application spends its time
  - Logical understanding of the code recommended
  - But actual measurement (profiling) required, too
- Make sure you use all the resources you have access to
  - All cores on the local CPU(s)
  - All the nodes you have access to
- You code is likely memory-bound
  - Look for register and Cache-friendly optimization options first
- Modern CPUs do vector math
  - Help the compiler generate full-width vector instructions
- A better algorithm may give you an even better speedup
  - See if anyone else is doing something similar (and consider libraries)